MLOps Heart Disease — Final Submission

End-to-End ML Model Development, CI/CD, and Production Deployment

Uday Singh — 2025cs05022

MLOps (S2-25_AMLCSZG523), Assignment I

Submission Overview
Submission Links
Student Details
1. Problem Statement & Dataset
2. Setup & Install Instructions
3. Data Acquisition & EDA (5 marks)
4. Feature Engineering & Model Development (8 marks)
5. Experiment Tracking — MLflow (5 marks)
6. Model Packaging & Reproducibility (7 marks)
7. CI/CD Pipeline & Automated Testing (8 marks)
8. Containerization (5 marks)
9. Production Deployment (7 marks)
10. Monitoring & Logging (3 marks)
11. Architecture Diagram
12. Repository Layout
13. Deliverables Checklist
14. Production-Readiness
15. Quick-Reference Links

Submission Overview

This document is the consolidated submission for the MLOps Assignment I. It contains every required artifact, links to source code, and a link to the end-to-end demonstration video.

Submission Links

Artifact	Location
Public GitHub	https://github.com/uday121212/mlops-heart-disease

Artifact	Location
repository	
Demo video (Google Drive)	https://drive.google.com/file/d/1jNtpcas45VublVSxIq0Rsypwvtw1Fdxu/vie usp=sharing
CI/CD workflow	https://github.com/uday121212/mlops-heart-disease/actions
Final report (this file)	`docs/SUBMISSION.pdf` / `docs/SUBMISSION.docx`

Student Details

- **Name:** Uday Singh
- **ID:** 2025cs05022
- **Course:** MLOps (S2-25_AMLCSZG523)
- **Assignment:** I — End-to-End ML Model Development, CI/CD, and Production Deployment
- **Total Marks:** 50

---

1. Problem Statement & Dataset

Problem: Build a binary classifier that predicts the risk of heart disease from patient health features, and deploy it as a cloud-ready, monitored REST API.

Dataset: UCI Heart Disease (Cleveland) — 303 rows × 14 attributes. Source: https://archive.ics.uci.edu/ml/datasets/heart+disease.

Feature	Type	Description
age	numeric	Patient age (years)
sex	categorical (0/1)	1 = male, 0 = female
cp	categorical (0–3)	Chest pain type
trestbps	numeric	Resting blood pressure (mm Hg)
chol	numeric	Serum cholesterol (mg/dl)
fbs	categorical	Fasting blood sugar > 120 mg/dl
restecg	categorical	Resting ECG result
thalach	numeric	Max heart rate achieved
exang	categorical	Exercise-induced angina
oldpeak	numeric	ST depression
slope	categorical	Slope of peak ST segment
ca	numeric	# major vessels coloured by fluoroscopy

Feature	Type	Description
thal	categorical	Thalassemia indicator
target	binary	1 = heart disease, 0 = none

---

2. Setup & Install Instructions

Clone and bootstrap a clean environment:

```
git clone https://github.com/uday121212/mlops-heart-disease.git
cd mlops-heart-disease
python -m venv .venv && source .venv/bin/activate
pip install -r requirements-dev.txt
```

Run the full pipeline locally:

```
python -m src.download_data    # fetch + clean dataset
python -m src.train            # train, GridSearchCV, log to MLflow,
        save model
pytest -v                      # unit + API tests
uvicorn src.api:app --reload   # serve API on http://localhost:8000
```

Run inside Docker:

```
docker build -t heart-disease-api:latest .
docker run -p 8000:8000 heart-disease-api:latest
```

Full stack (API + Prometheus + Grafana) via Compose:

```
docker compose up --build
```

Kubernetes deployment:

```
kubectl apply -f k8s/
# or, via Helm
helm install heart-api ./helm/heart-api
```

---

3. Data Acquisition & EDA (5 marks)

`src/download_data.py` fetches `processed.cleveland.data` from the UCI repository and produces `data/heart_disease_clean.csv`. Cleaning steps:

- Replace `?` markers with `NaN`.
- Median-impute `ca` and `thal`.
- Binarize the target (`num` 0 → 0, 1–4 → 1).

EDA highlights (`notebooks/01_eda.ipynb`):

- **Class balance:** ~54 % no-disease / 46 % disease — no resampling required.
- **Age:** mean ≈ 54, std ≈ 9.
- **Top correlations with target:** `cp`, `thalach` (negative), `oldpeak`, `ca`, `exang`.
- **No severe multicollinearity** between features.

Artifacts produced: histograms (`reports/hist_*.png`), correlation heatmap (`reports/corr_heatmap.png`), class-balance plot (`reports/class_balance.png`).

---

4. Feature Engineering & Model Development (8 marks)

Preprocessing pipeline (a single `ColumnTransformer`, shipped with the model so train/serve preprocessing is identical):

- **Numeric features** (`age`, `trestbps`, `chol`, `thalach`, `oldpeak`, `ca`) → `StandardScaler`.
- **Categorical features** (`sex`, `cp`, `fbs`, `restecg`, `exang`, `slope`, `thal`) → `OneHotEncoder(handle_unknown="ignore")`.

Models compared with 5-fold stratified `GridSearchCV` (scoring = ROC-AUC):

Model	Hyper-parameter grid
Logistic Regression	`C ∈ {0.1, 1, 10}`, `penalty ∈ {l1, l2}`, `solver=liblinear`
Random Forest	`n_estimators ∈ {200, 400}`, `max_depth ∈ {None, 5, 10}`, `min_samples_split ∈ {2, 5}`

The model with the highest CV ROC-AUC on the train split is selected, re-fit on the full training set, evaluated on a held-out test set, and persisted to `models/heart_disease_model.joblib`. Final metrics are written to `reports/metrics.json` (accuracy, precision, recall, F1, ROC-AUC).

---

5. Experiment Tracking — MLflow (5 marks)

Every grid-search run is tracked in MLflow with:

- **Parameters** — best hyper-parameters per estimator.
- **Metrics** — CV ROC-AUC and held-out test accuracy / precision / recall / F1 / ROC-AUC.
- **Artifacts** — the full fitted `Pipeline` via `mlflow.sklearn.log_model`, plus EDA plots.

Tracking URI is `file://./mlruns`. Browse with:

```
mlflow ui --backend-store-uri ./mlruns
```

Screenshots: `screenshots/mlflow-runs.png`, `screenshots/mlflow-experiment.png`.

---

6. Model Packaging & Reproducibility (7 marks)

- `requirements.txt` — pinned runtime dependencies.
- `requirements-dev.txt` — adds testing/lint tooling.
- Serialized model is the **full `Pipeline`** (preprocessor + estimator), guaranteeing no train/serve skew.
- The same `build_pipeline` factory is used by training and tests, so preprocessing is fully reproducible from a clean checkout.

---

7. CI/CD Pipeline & Automated Testing (8 marks)

`.github/workflows/ci.yml` runs on every push and PR. Two jobs:

1. **lint-test-train**
   - `flake8` lint
   - `python -m src.download_data`
   - `python -m src.train`
   - `pytest --cov`
   - Uploads `models/heart_disease_model.joblib` and coverage report.
2. **docker-build**
   - Downloads the trained-model artifact
   - Builds the image with Buildx
   - Runs the container and smoke-tests `/health` and `/predict`.

The pipeline fails on any lint, test, or training error, with the failing logs surfaced in the Actions UI. Screenshot: `screenshots/ci.png`.

Unit tests live under `tests/`:

- `test_preprocessing.py` — schema, pipeline shape, encoding.
- `test_training.py` — train smoke-test, persisted-model round-trip.
- `test_api.py` — FastAPI `TestClient` for `/health` and `/predict`.

---

8. Containerization (5 marks)

`Dockerfile` (multi-stage, `python:3.11-slim`):

- Installs pinned dependencies.
- Copies `src/` and `models/`.
- Runs as a **non-root** `appuser`.
- Exposes port 8000.
- Includes a `HEALTHCHECK` that calls `/health`.
- Launches `uvicorn src.api:app --host 0.0.0.0 --port 8000`.

`docker-compose.yml` orchestrates the API plus a Prometheus and Grafana container with a pre-provisioned data source and dashboard.

Sample request:

```
curl -X POST http://localhost:8000/predict \
  -H "Content-Type: application/json" \
  -d '{"age":63,"sex":1,"cp":3,"trestbps":145,"chol":233,
       "fbs":1,"restecg":0,"thalach":150,"exang":0,"oldpeak":2.3,
       "slope":0,"ca":0,"thal":1}'
```

Response (example):

```
{ "prediction": 1, "probability": 0.87 }
```

---

9. Production Deployment (7 marks)

Kubernetes manifests in `k8s/`:

- `deployment.yaml` — 2 replicas; readiness/liveness probes on `/health`; CPU/memory requests + limits; Prometheus scrape annotations.
- `service.yaml` — `LoadBalancer` on port 80 → container 8000.
- `ingress.yaml` — optional NGINX ingress on host `heart-api.local`.

Helm chart at `helm/heart-api/` parameterizes:

- Image repo + tag
- Replica count
- Service type
- Ingress host/TLS
- HorizontalPodAutoscaler thresholds

Quick deploy:

```
docker build -t heart-disease-api:latest .
kubectl apply -f k8s/
kubectl port-forward svc/heart-api 8080:80
curl http://localhost:8080/health
```

Verification screenshots: `screenshots/k8s-pods.png`, `screenshots/k8s-service.png`, `screenshots/predict.png`, `screenshots/swagger-ui.png`.

---

10. Monitoring & Logging (3 marks)

- The FastAPI app emits structured `logging` records per request (timestamp, level, route, status, latency, prediction).
- `prometheus-fastapi-instrumentator` exposes default HTTP metrics on `/metrics` (request count by handler/status, latency histograms, in-progress gauges).
- Prometheus scrape config: `monitoring/prometheus.yml`.
- Grafana dashboard auto-provisioned from `monitoring/grafana/dashboards/heart-api.json` with panels for request-rate, p95 latency, and total requests.

Screenshots: `screenshots/grafana.png`, `screenshots/prometheus.png`, `screenshots/api-metrics.png`.

---

11. Architecture Diagram

```
                  ┌──────────────────────┐
                  │ UCI Heart Disease    │
                  │ (Cleveland CSV)      │
                  └──────────────────────┘
                           │
                           │ src/download_data.py
                           ▼
                ┌──────────────────────┐
                │ data/heart_disease_*.csv │
                └──────────────────────┘
                           │
                           │ src/train.py
                           │ (GridSearchCV + MLflow)
                           ▼
┌──────────┐    ┌──────────────────────┐
│ MLflow   │◄──►│ models/              │
│ tracking │    │ heart_disease_model. │
│ (./mlruns)│   │   joblib (Pipeline)  │
└──────────┘    └──────────────────────┘
                           │
                           │ FastAPI (src/api.py)
                           ▼
                  ┌──────────────────────┐
                  │ Docker image         │
                  │ heart-disease-api    │
                  └──────────────────────┘
                           │
                           │ kubectl / helm
                           ▼
┌──────────────────────────────────────────┐
```

```
│ Kubernetes (Deployment + Service + Ingress) │
│     │              │              │          │
│     ▼              ▼              ▼          │
│  /predict       /metrics       /health       │
└──────────────────────────────────────────────┘
          │                              │
          │ scrape                       │ logs
          ▼                              ▼
      Prometheus ──────► Grafana   stdout / kubectl logs
```

---

12. Repository Layout

Path	Purpose
`src/`	Source: `download_data.py`, `preprocessing.py`, `train.py`, `api.py`, `inference.py`
`tests/`	Pytest suite (preprocessing, training, API)
`notebooks/`	`01_eda.ipynb`, `02_training.ipynb`, `03_inference.ipynb`
`data/`	Cleaned + raw CSVs
`models/`	Persisted joblib pipeline
`reports/`	Metrics JSON + figures
`mlruns/`	MLflow tracking store
`Dockerfile` / `docker-compose.yml`	Container build + full stack
`k8s/`	Kubernetes manifests
`helm/heart-api/`	Helm chart
`monitoring/`	Prometheus + Grafana config + dashboard
`.github/workflows/ci.yml`	CI/CD pipeline
`screenshots/`	Deployment, monitoring, CI screenshots
`video/`	README pointing to the demo video on Google Drive
`docs/`	Architecture notes + this report

---

13. Deliverables Checklist

Required Deliverable	Location	Status
Code + Dockerfile + requirements.txt	repo root	✅
Cleaned dataset + download script	`data/`, `src/download_data.py`	✅

Required Deliverable	Location	Status
Jupyter notebooks (EDA, training, inference)	`notebooks/`	✅
Unit tests	`tests/`	✅
GitHub Actions workflow YAML	`.github/workflows/ci.yml`	✅
Deployment manifests + Helm chart	`k8s/`, `helm/heart-api/`	✅
Screenshot folder	`screenshots/`	✅
Final written report (this file)	`docs/SUBMISSION.pdf` / `.docx`	✅
Short demo video (end-to-end pipeline)	Google Drive	✅
Deployed API access instructions	section 8–9 above	✅

14. Production-Readiness

- [x] All scripts run from a clean venv via `requirements-dev.txt`.
- [x] Model serves correctly inside Docker (smoke-tested in CI).
- [x] CI fails fast on lint / test / training errors and surfaces logs.
- [x] Preprocessing bundled with the model → no train/serve skew.
- [x] `/health`, `/metrics`, and per-request logs exposed.
- [x] Kubernetes probes, resource limits, non-root container.
- [x] Prometheus + Grafana monitoring stack provisioned.

---

15. Quick-Reference Links

- **Public repository:** https://github.com/uday121212/mlops-heart-disease
- **CI runs:** https://github.com/uday121212/mlops-heart-disease/actions
- **Demo video:** https://drive.google.com/file/d/1jNtpcas45VublVSxIq0Rsypwvtw1Fdxu/view?usp=sharing

End of submission document.